

Memoria: Arquitectura y Modelado

David Cendejas Rodríguez y Ángel Jiménez Izquierdo
Borrador para reunión con experto

23 de abril de 2026

Resumen

Este documento detalla la justificación teórica, el tratamiento de datos y la estructura interna utilizada en el desarrollo de la red neuronal predictiva para la inferencia de parámetros de síntesis FM.

Índice

1. Arquitectura y Modelado de la Red Neuronal	2
1.1. Exploración de Paradigmas: Clasificación vs. Regresión	2
1.1.1. Enfoque de Clasificación	2
1.1.2. Enfoque de Regresión	2
1.2. El Problema de la Inyectividad y la Necesidad de una Arquitectura Compleja	3
1.2.1. Prototipo 4: Red Convolutiva de Doble Cabeza	4
1.2.2. Prototipo 4: Función de Pérdida Híbrida	5
1.3. Prototipo 5: Modelo Final y Estudio de Posibilidades	6
1.3.1. Expansión Paramétrica: Evolventes y Generación Aleatoria del Dataset	6
1.3.2. Experimentación	7
1.3.3. Pipelines y Preguntas de investigación	7
1.4. Justificación del Stack Tecnológico y Librerías Utilizadas	7

1. Arquitectura y Modelado de la Red Neuronal

1.1. Exploración de Paradigmas: Clasificación vs. Regresión

1.1.1. Enfoque de Clasificación

La clasificación es el paso previo natural a la búsqueda de un modelo de inferencia de parámetros. De esta manera, se puede comprobar de forma práctica que la red neuronal procesa el audio adecuadamente y es capaz de conservar su información acústica fundamental.

El **Prototipo 1** es una prueba de concepto para la primera aproximación al aprendizaje automático aplicado al audio. En él se utiliza un dataset reducido, construido con la librería NumPy, de alrededor de 50 archivos *.wav* de un segundo de duración. Estos audios se generan sintéticamente con frecuencias aleatorias y formas de onda básicas (*sine*, *square*, *sawtooth*, *triangle* y *noise*).

En la fase de preprocesamiento y entrenamiento, desarrollada utilizando TensorFlow, se opta por no aportar a la red formas de audio crudo. En su lugar, se extraen las características calculando los Coeficientes Cepstrales en las Frecuencias de Mel (MFCC) de cada muestra. El modelo tiene el objetivo de clasificar y asociar la matriz de coeficientes con la etiqueta correspondiente a su tipo de onda. A pesar de esto, al plantear una arquitectura básica no convolucional, la red neuronal carece de los mecanismos para realizar una interpretación correcta de los patrones bidimensionales de los MFCC. Como consecuencia, el modelo arroja una precisión muy baja.

El **Prototipo 2** se basa en el **Prototipo 1**. Sigue atendiendo al objetivo de la clasificación, pero tiene características que difieren del anterior. Esta vez, los audios se convierten a espectrogramas *.png* mediante el uso de la librería de *librosa*. Esto conlleva un menor coste computacional y de almacenamiento. Al tratarse realmente de imágenes, se pueden utilizar modelos preentrenados de visión, encargados de clasificar imágenes.

Para el entrenamiento, se hace uso de **ResNet34 preentrenada** vía *fastAI*. Para su adaptación, se utilizan **DataBlocks**, una herramienta de *fastAI* mediante la cual se especifican que las entradas son las imágenes (espectrogramas) y las salidas son las categorías (tipo de onda). Asimismo, se etiqueta obteniendo la categoría a partir del nombre archivo y se divide destinando un 80 % de las imágenes para entrenar y un 20 % para validación.

Como resultado de utilizar una red convolucional y la arquitectura externa **ResNet34**, el modelo obtuvo una buena precisión, especialmente identificando ondas puras dentro del rango entrenado.

1.1.2. Enfoque de Regresión

Al plantear un salto hacia la síntesis paramétrica continua, fue necesario tomar una decisión de diseño fundamental: la elección del motor de síntesis. Tras la evaluación de distintas alternativas, se optó por centrar el proyecto en la síntesis por Modulación de Frecuencia (FM). Esta técnica presenta ventajas claras para el aprendizaje automático. A diferencia de la síntesis aditiva, que requiere decenas de osciladores simultáneos, la síntesis FM es muy eficiente, consiguiendo generar timbres complejos a partir de un número reducido de parámetros. De esta forma, se puede comenzar utilizando, por ejemplo, la frecuencia portadora, el ratio y el índice de modulación, y luego ampliar el número de

variables para conseguir sonidos más complejos. Estas características acotan la dimensionalidad del problema predictivo, haciéndolo viable para la red neuronal.

Si bien con la clasificación queda validada la extracción de características acústicas, una vez tomada esta última decisión, deja de ser útil. Para la inferencia de parámetros de la síntesis FM, resulta obligatoria la evolución hacia una arquitectura de regresión.

Para el **Prototipo 3** se genera un dataset mediano de aproximadamente 15000 muestras, construido mediante un barrido de parámetros en un sintetizador FM de **pyo**. Para la reducción de la carga computacional y la optimización del espacio, se sustituye **librosa** por la librería de *torchaudio*. Esta permite transformar los archivos de audio (*.wav*) en tensores de espectrogramas (*.pt*), asociados mediante el uso de un (*.csv*) a sus valores numéricos. El 80 % del conjunto de datos se destina a entrenamiento y el otro 20 % a validación.

El entrenamiento se implementa mediante **PyTorch**. Se diseña una arquitectura convolucional propia (SmallCNNRegressor). La red neuronal se compone de dos bloques principales: uno de extracción de características y una cabeza de regresión.

El primer bloque recibe los espectrogramas en forma de tensores de un único canal (lo que equivaldría a imágenes en escala de grises) y los procesa en otros tres bloques convolucionales. Cada uno de estos aplica kernels de tamaño 3x3 (o núcleos convolucionales) que se deslizan sobre la matriz del espectrograma para extraer características espaciales, detectando así patrones visuales en la representación espectral del sonido. A esto le sigue una función de activación no lineal **ReLU** y una capa de reducción espacial (**Max Pooling**). El último bloque convolucional termina en una capa de agrupamiento (**Adaptive Average Pooling**), que comprime los mapas de características a una matriz 4x4.

El siguiente paso es la cabeza de regresión, en donde la información espacio-temporal del espectrograma (extraída en el bloque anterior), se pasa a un vector de una única dimensión y atraviesa dos capas completamente conectadas (**Perceptrón Multicapa**). La primera, comprime la información a 128 neuronas, y la segunda, saca el vector por una dimensión de salida de tamaño 3, que se corresponde con los tres parámetros del sintetizador: frecuencia portadora, ratio y el índice de modulación.

El modelo optimiza el proceso de los espectrogramas mediante el algoritmo Adam, utilizando el Error Cuadrático Medio (MSELoss) como función de pérdida. Y, en cuanto al ciclo de entrenamiento, este se configuró con una duración de 5 épocas (*epochs*). En este prototipo se decidió no hacer una búsqueda extensa de hiperparámetros al tratarse de un prototipo intermedio, con un fin de experimentación inicial con la regresión.

A pesar de esto, se puede observar una precisión muy limitada en los resultados experimentales. Se consigue mantener un MSE bajo en el *Ratio* e *Index*, pero el error en el *Carrier* resulta desproporcionado, debido a operar en la escala de miles de hercios. De este prototipo se puede sacar la conclusión de que el minimizar el error matemático sobre los parámetros no garantiza la similitud acústica del sonido generado.

1.2. El Problema de la Inyectividad y la Necesidad de una Arquitectura Compleja

El **Prototipo 3** contiene una limitación fundamental muy relevante en la síntesis FM: su inyectividad es nula. Esto quiere decir que diferentes combinaciones de parámetros pueden dar lugar a sonidos idénticos. Un ejemplo, radical pero que explica muy bien este resultado, es un caso en el que el índice de modulación sea 0. En ese caso, no es verdaderamente relevante el valor del ratio, sin embargo, la función de pérdida, implementada

con **MSE**, penaliza gravemente que este difiera del original. Este obstáculo hace que se tenga que replantear la función de pérdida.

1.2.1. Prototipo 4: Red Convolutiva de Doble Cabeza

El **Prototipo 4** se diseña para solucionar el problema de la inyectividad, tratando de asegurar que la inferencia de parámetros se corresponda con la fidelidad acústica. Este modelo se basa en una arquitectura convolutiva de doble cabeza (**Encoder-Decoder**).

En cuanto al preprocesamiento del conjunto de datos, se implementa una mejora para evitar que la diferencia tan grande entre las escalas de diferentes parámetros (como la frecuencia portadora frente al ratio) comprometa el cálculo del gradiente. Los tensores se normalizan estadísticamente (**Z-score**), restando la media y dividiendo por la desviación estándar de cada parámetro.

La topología de la red (CNNRegressor4), consta de un procesamiento dividido en varias fases:

Primero se pasa por un Codificador (**Encoder**), el cual consta de una secuencia de tres bloques convolutivos que convierten el espectrograma inicial en una representación comprimida, extrayendo así sus características fundamentales. Para ello, cada bloque aplica núcleos (kernels) de 3x3. Posteriormente, pasa por una capa de normalización por lotes (Batch Normalization), que estabiliza y acelera el entrenamiento, y una activación no lineal ReLU. Finalmente, se aplica una reducción espacial en cada bloque (Max Pooling). Con todo esto, la matriz se ha reducido a una octava parte de su tamaño original y la profundidad de la red ha incrementado de 1 a 128 canales. Como resultado, se obtiene una representación comprimida rica en características acústicas.

Al salir del **Encoder**, el flujo de datos atraviesa lo que se denomina como cuello de botella (**Bottleneck**). Este bloque convolutivo no aplica una reducción espacial, manteniendo estables las dimensiones de la matriz. Su función es duplicar el número de canales (profundidad del tensor), pasando de 128 a 256 canales. Esto se hace con la finalidad de mezclar los patrones que el **Encoder** ha extraído previamente, creando una representación densa y unificada.

Después se pasa por la fase de Inferencia Numérica, donde ocurre la regresión de los parámetros del sintetizador FM (frecuencia portadora, ratio e índice). Se aplica una capa de agrupamiento global (Adaptive Average Pooling) ya que los parámetros describen el sonido completo y no un punto temporal concreto en el espectrograma. Para ello, se calcula la media de todas las posiciones de cada canal, quedándose con un único número como resumen por canal. Finalmente, el vector unidimensional, resultado del proceso anterior, atraviesa un **Perceptrón Multicapa**, que lo proyecta hacia las tres neuronas de salida.

Por último, se encuentra la fase de Reconstrucción (**Decoder**), la cual se encarga de reconstruir el espectrograma original a partir de la representación comprimida del **Bottleneck**. Como el **Encoder** comprime tres veces, el **Decoder** descomprime tres veces. Para ello, utiliza tres capas convolutivas transpuestas (ConvTranspose2d).

El motivo para utilizar el **Decoder** es como método de regularización forzada del **Encoder**. En el caso en el que el codificador borre información importante, como armónicos o patrones temporales, que no afecte directamente a los parámetros de la regresión, el (**Decoder**) no será capaz de redibujar correctamente el espectrograma, disparando así el error en la función de pérdida híbrida. Aparte, si el **Bottleneck** no guarda suficiente información, ocurrirá lo mismo. Todo esto fuerza, mediante el cálculo de gradientes, a que las capas iniciales aprendan a conservar el timbre y la representación del sonido.

1.2.2. Prototipo 4: Función de Pérdida Híbrida

Con el objetivo de solucionar el problema de la inyectividad, se consolida el uso de utilizando Función de Pérdida Híbrida, abandonando el Error Cuadrático Medio (MSE). Esta función evalúa la pérdida de dos formas simultáneamente, dándole prioridad a la similitud y convergencia de espectrogramas (pérdida y convergencia espectral) y dejando en segundo plano la similitud de los parámetros (pérdida paramétrica).

Por un lado tenemos la **Pérdida y Convergencia Espectral**. Estas funciones son las principales para la composición del error, teniendo mucho más peso que la pérdida paramétrica, y estando encargadas de asegurar que un sonido generado sea acústicamente similar al original.

La **pérdida espectral** se calcula mediante el Error Cuadrático Medio (**L1 Loss**), el cual mide la diferencia directa entre espectrogramas (píxel a píxel) en escala logarítmica (dB), siendo la métrica de mayor peso para el error. Con esto se garantiza unos picos de energía y formas generales que coincidan en tiempo y frecuencia.

En cuanto a la **convergencia espectral**, es un refuerzo (tiene un peso bajo) para la pérdida espectral, que compara las magnitudes de forma relativa con la norma de Frobenius, capturando la estructura global del espectro (como los armónicos y la energía). Estas características son muy relevantes para encontrar una representación fiel al timbre original.

Por último se encuentra la **Pérdida Paramétrica**, aplicada mediante **SmoothL1**. Esta penaliza las diferencias en los parámetros del sintetizador, por lo que no se elimina totalmente la regresión numérica de los parámetros, pero se le un peso considerablemente bajo.

La ponderación de los pesos en la función de pérdida híbrida es fundamental para el correcto aprendizaje del Modelo y también para solucionar el problema de la inyectividad. Se le otorga un peso completo (1.0) a la pérdida espectral, un peso medio (0.5) a la convergencia espectral y un peso muy bajo (0.05) a la pérdida paramétrica. De esta manera, en el caso de que la red infiera una serie de parámetros que difieran considerablemente de los parámetros originales, si sus espectrogramas son muy similares (y, por tanto, su timbre también lo es), el error será mínimo. El modelo debe memorizar números exactos, si no que analiza la imagen del sonido, su espectrograma.

1.3. Prototipo 5: Modelo Final y Estudio de Posibilidades

Se adjunta la Figura 1 con el objetivo de facilitar la comprensión del modelo del Prototipo 5. El trabajo queda dividido en 5 fases: generación de audio, conversión a tensores, preparación del dataset, entrenamiento (con múltiples opciones de arquitectura) e inferencia paramétrica.

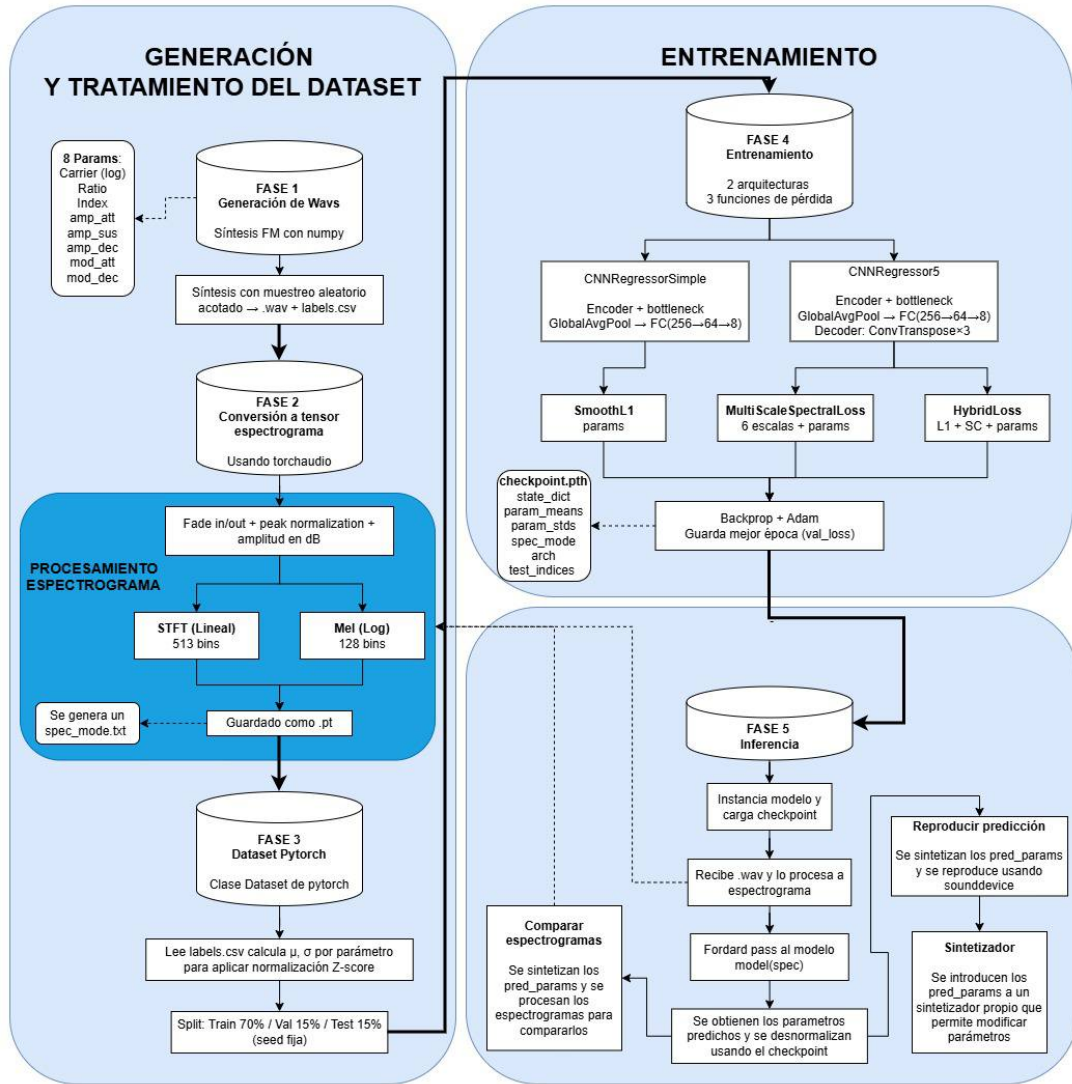


Figura 1: Flujo de trabajo del Prototipo 5: desde la síntesis matemática hasta la inferencia y posterior aplicación en la interfaz de usuario.

1.3.1. Expansión Paramétrica: Evolventes y Generación Aleatoria del Dataset

Las predicciones del **Prototipo 4** consiguen replicar detalladamente el timbre. Sin embargo, los sonidos generados en esta etapa están muy alejados de las aplicaciones reales del **Sound Matching**. Por esta razón, se propone un nuevo modelo en el que se explora ampliar la dimensionalidad de 3 a 8 parámetros. A las variables fundamentales (frecuencia portadora, ratio e índice), se le añaden las envolventes temporales: la envolvente de amplitud (*attack*, *decay* y *sustain*) y la envolvente del índice de modulación (*attack* y *decay*).

Son precisamente estas características del **Prototipo 5**, por las que es necesario una nueva forma de generación del dataset. La ampliación de 3 a 8 parámetros hace computacional inviable generar el dataset mediante un barrido de combinaciones posibles. Para dar solución a esta limitación, se genera el dataset por muestreo aleatorio.

El sistema genera un número (N) de muestras de audio con valores continuos aleatorios, en base a unos rangos de valores específicos para cada parámetro.

En primer lugar, se sintetizan los N archivos *.wav* mediante la librería **NumPy** y la formulación matemática de la FM. Se define una frecuencia de muestreo (*sample rate*) de 44100 Hz y un tiempo de duración de audio de 2 segundos. Se incrementa el tiempo de duración (de 1 a 2 segundos) con el fin de asegurar un margen temporal necesario para analizar y percibir las variaciones en el sonido provocadas por las envolventes. Estas envolventes se generan sobre un vector de tiempo unificado.

Posteriormente, los audios generados *.wav* se convierten a tensores nativos de *PyTorch*. Se cargan mediante *torchaudio* y se les aplica un suavizado de amplitud en los extremos (*fade-in* y *fade-out*) con el objetivo de evitar chasquidos o artefactos indeseados. Finalmente, se aplican las dos transformaciones principales: de audio a espectrograma y de amplitud lineal a logarítmica (dB). Esta función aplica una normalización de pico, calcula la STFT, extrae la magnitud y realiza su conversión a decibelios.

1.3.2. Experimentación

1.3.3. Pipelines y Preguntas de investigación

1.4. Justificación del Stack Tecnológico y Librerías Utilizadas